МГТУ им Н Э Баумана

Кафедра ИУ5

Парадигмы и конструкции языков программирования

Лабораторная работа 6

Простой Telegram бот с кнопками

Вариант 17

Выполнил _____

Группа _____

Проверил _____

Москва 2026

Задание

Разработать простого Telegram бота на Python, использующего функциональность создания кнопок.

Условие работы: https://github.com/ugapanyuk/BKIT_2021/wiki/lab_bot1

Решение

Учебный бот показывает reply клавиатуру с кнопками Лабораторные, Вариант и Помощь. Функция reply формирует ответ и клавиатуру, а отдельный модуль выполняет getUpdates и sendMessage через стандартный urllib. Секретный токен читается из TELEGRAM_BOT_TOKEN и не записывается в исходники.

Использован Python 3.12.14. Команды выполнялись из папки работы. Исходные файлы и полный протокол приложены рядом с отчётом.

Текст программы

Файл labs/06_bot/bot.py

```
KEYBOARD = {'keyboard': [['Лабораторные', 'Вариант'], ['Помощь']],
            'resize_keyboard': True}


def reply(user, text):
    answers = {
        '/start': 'Привет! Выберите кнопку учебного бота.',
        'Лабораторные': 'Темы: основы Python, ООП, функции, тесты и Telegram-боты.',
        'Вариант': 'Номер варианта: 17.',
        'Помощь': 'Нажмите кнопку. Команда /start показывает меню.',
        '/help': 'Нажмите кнопку. Команда /start показывает меню.',
    }
    return answers.get(text, 'Неизвестная команда. Выберите кнопку.'), KEYBOARD
```

Файл labs/06_bot/main.py

```
import sys
from bot import reply
from telegram_api import run_bot


if __name__ == '__main__':
    if '--demo' in sys.argv:
        for message in ['/start', 'Лабораторные', 'Вариант', 'Помощь', '???']:
            answer, keyboard = reply(1, message)
            print('Пользователь:', message)
            print('Бот:', answer)
        print('Кнопки:', keyboard['keyboard'])
    else:
        run_bot(reply)
```

Файл labs/06_bot/telegram_api.py

```
import json
import os
from urllib.request import Request, urlopen
from urllib.error import URLError


def api_call(token, method, payload):
    data = json.dumps(payload).encode('utf-8')
    request = Request(f'https://api.telegram.org/bot{token}/{method}', data=data,
                      headers={'Content-Type': 'application/json'})
    try:
        with urlopen(request, timeout=40) as response:
            answer = json.load(response)
    except (URLError, TimeoutError, ValueError):
        # Не выводим URL ошибки: он содержит секретный токен.
        raise RuntimeError('Не удалось обратиться к Telegram API') from None
    if not answer.get('ok'):
        raise RuntimeError('Telegram API отклонил запрос')
    return answer['result']
```

```
def run_bot(reply):
    token = os.environ.get('TELEGRAM_BOT_TOKEN')
    if not token:
        raise SystemExit('Задайте TELEGRAM_BOT_TOKEN или запустите main.py --demo')
    offset = 0
    print('Бот запущен. Для остановки нажмите Ctrl+C.')
    try:
        while True:
            updates = api_call(token, 'getUpdates', {'offset': offset, 'timeout': 30,
                                                     'allowed_updates': ['message']})
            for update in updates:
                message = update.get('message', {})
                if 'text' in message:
                    key = (message['chat']['id'], message['from']['id'])
                    text, keyboard = reply(key, message['text'])
                    payload = {'chat_id': message['chat']['id'], 'text': text}
                    if keyboard:
                        payload['reply_markup'] = keyboard
                    api_call(token, 'sendMessage', payload)
                offset = update['update_id'] + 1
    except KeyboardInterrupt:
        print('Бот остановлен.')
    except RuntimeError as error:
        raise SystemExit(str(error)) from None
```

Файл labs/06_bot/test_bot.py

```
import unittest
from unittest.mock import patch
from bot import reply
from telegram_api import api_call


class BotTests(unittest.TestCase):
    def test_menu_has_buttons(self):
        text, keyboard = reply(1, '/start')
        self.assertIn('Выберите', text)
        self.assertIn(['Лабораторные', 'Вариант'], keyboard['keyboard'])

    def test_variant_and_unknown_message(self):
        self.assertIn('17', reply(1, 'Вариант')[0])
        self.assertIn('Неизвестная', reply(1, '???')[0])

    @patch('telegram_api.urlopen')
    def test_api_payload_with_mock(self, urlopen):
        response = urlopen.return_value.__enter__.return_value
        response.read.return_value = b'{"ok": true, "result": []}'
        self.assertEqual(api_call('test-token', 'getUpdates', {}), [])
```

Результаты выполнения

```
$ python main.py --demo
Пользователь: /start
Бот: Привет! Выберите кнопку учебного бота.
Пользователь: Лабораторные
Бот: Темы: основы Python, ООП, функции, тесты и Telegram-боты.
Пользователь: Вариант
Бот: Номер варианта: 17.
Пользователь: Помощь
Бот: Нажмите кнопку. Команда /start показывает меню.
Пользователь: ???
Бот: Неизвестная команда. Выберите кнопку.
Кнопки: [['Лабораторные', 'Вариант'], ['Помощь']]
```

Вывод локального запуска 1 часть 1

```
$ python -m unittest discover -v
test_api_payload_with_mock (test_bot.BotTests.test_api_payload_with_mock) ... ok
test_menu_has_buttons (test_bot.BotTests.test_menu_has_buttons) ... ok
test_variant_and_unknown_message
(test_bot.BotTests.test_variant_and_unknown_message) ... ok

----------------------------------------------------------------------
Ran 3 tests in 0.002s

OK
```

Вывод локального запуска 2 часть 1

Проверка результата

Локально проверены ответы меню, неизвестная команда и JSON ответ транспорта с Mock. Демонстрационный режим не подключается к Telegram. Для проверки живого бота нужно создать его через BotFather, задать токен и выполнить python main.py. Скриншоты переписки в Telegram в отчёт не включены, поскольку сетевой запуск без токена не выполнялся.